

TENSORORBIT EDA BOOTCAMP - STUDENT CODE CHEAT SHEET

Simple reference for the capstone: If you want to do this, use this code

Use `df` as your `DataFrame` name. Replace column names such as "Sales" or "Category" with the columns in your own dataset.

1. START: LOAD AND UNDERSTAND YOUR DATA

IF YOU WANT TO...	USE THIS CODE	WHAT IT DOES / REMEMBER
Import the main libraries	<pre>import pandas as pd import numpy as np import matplotlib.pyplot as plt</pre>	Run this first.
Load a CSV file	<pre>df = pd.read_csv("your_file.csv")</pre>	Change the file name to your dataset.
See the first 5 rows	<pre>df.head()</pre>	Quick look at the dataset.
See the last 5 rows	<pre>df.tail()</pre>	Useful for checking the end of the data.
Check rows and columns	<pre>df.shape</pre>	Output is (rows, columns).
See all column names	<pre>df.columns</pre>	Use the exact spelling when writing code.
Check data types and missing values	<pre>df.info()</pre>	Shows columns, data types and non-null counts.
Get basic statistics	<pre>df.describe()</pre>	Shows count, mean, std, min, quartiles and max.

2. CLEAN YOUR DATA

IF YOU WANT TO...	USE THIS CODE	WHAT IT DOES / REMEMBER
Check missing values	<pre>df.isnull().sum()</pre>	Shows how many missing values each column has.
Fill a missing number with the mean	<pre>df["Sales"] = df["Sales"].fillna(df["Sales"].mean())</pre>	Good for a numeric column when appropriate.
Fill a missing category	<pre>df["Customer_Type"] = df["Customer_Type"].fillna("Unknown")</pre>	Useful for text/category columns.
Remove rows with missing values	<pre>df.dropna(inplace=True)</pre>	Use only when removing those rows makes sense.
Count duplicate rows	<pre>df.duplicated().sum()</pre>	Tells you how many exact duplicates exist.
Remove duplicate rows	<pre>df.drop_duplicates(inplace=True)</pre>	Keeps one copy and removes duplicates.
Change a column to numeric	<pre>df["Quantity"] = df["Quantity"].astype(int)</pre>	Use only when the values can be converted.
Convert a date column	<pre>df["Order_Date"] = pd.to_datetime(df["Order_Date"])</pre>	Lets you work with year/month/day.

TENSORORBIT EDA BOOTCAMP - STUDENT CODE CHEAT SHEET

Page 2 - Explore and analyse

3. SELECT, FILTER AND SORT

IF YOU WANT TO...	USE THIS CODE	WHAT IT DOES / REMEMBER
Select one column	<code>df["Sales"]</code>	Returns one column.
Select several columns	<code>df[["Category", "Sales", "Profit"]]</code>	Put the column names inside a list.
Show rows where Sales is above 1000	<code>df[df["Sales"] > 1000]</code>	Change the number to your own condition.
Show one category only	<code>df[df["Category"] == "Technology"]</code>	Text values need quotation marks.
Use two conditions	<code>df[(df["Sales"] > 1000) & (df["Profit"] > 0)]</code>	& means AND. Put each condition in brackets.
Sort from highest to lowest	<code>df.sort_values("Sales", ascending=False)</code>	False = descending/highest first.
Sort from lowest to highest	<code>df.sort_values("Sales", ascending=True)</code>	True = ascending/lowest first.
Count each category	<code>df["Category"].value_counts()</code>	Useful for categories, regions, customer types, etc.

4. BUSINESS ANALYSIS WITH PANDAS

IF YOU WANT TO...	USE THIS CODE	WHAT IT DOES / REMEMBER
Find average sales	<code>df["Sales"].mean()</code>	Mean = average.
Find median sales	<code>df["Sales"].median()</code>	Median = middle value.
Find minimum and maximum	<code>df["Sales"].min()</code> <code>df["Sales"].max()</code>	Useful for ranges and unusual values.
Find total sales	<code>df["Sales"].sum()</code>	Adds all values in the column.
Find sales by category	<code>df.groupby("Category")["Sales"].sum()</code>	Groups categories and adds their sales.
Find average profit by region	<code>df.groupby("Region")["Profit"].mean()</code>	Change the group and measurement as needed.
Find several statistics by group	<code>df.groupby("Category")["Sales"].agg(["sum", "mean", "count"])</code>	Useful when you want more than one result.
Show best group first	<code>df.groupby("Category")</code> <code>["Sales"].sum().sort_values(ascending=False)</code>	Very useful for business rankings.

5. NUMPY: QUICK STATISTICS

IF YOU WANT TO...	USE THIS CODE	WHAT IT DOES / REMEMBER
Calculate mean	<code>np.mean(df["Sales"])</code>	Average value.
Calculate median	<code>np.median(df["Sales"])</code>	Middle value.
Calculate standard deviation	<code>np.std(df["Sales"])</code>	Shows how spread out the values are.
Find quartiles	<code>np.percentile(df["Sales"], [25, 50, 75])</code>	25%, median, and 75% points.

TENSORORBIT EDA BOOTCAMP - STUDENT CODE CHEAT SHEET

Page 3 - Relationships, outliers and charts

6. CORRELATION AND OUTLIERS

IF YOU WANT TO...	USE THIS CODE	WHAT IT DOES / REMEMBER
Check correlation between numeric columns	<code>df[["Sales", "Profit", "Quantity"]].corr()</code>	Values closer to 1 or -1 show stronger relationships. Correlation does not prove causation.
Find Q1 and Q3	<code>Q1 = df["Sales"].quantile(0.25)</code> <code>Q3 = df["Sales"].quantile(0.75)</code>	First step for IQR outlier checking.
Calculate IQR	<code>IQR = Q3 - Q1</code>	<code>IQR = Q3 - Q1</code> .
Find possible outliers with IQR	<code>outliers = df[(df["Sales"] < Q1 - 1.5*IQR) (df["Sales"] > Q3 + 1.5*IQR)]</code> <code>outliers</code>	Shows unusually low or high values.

7. CHOOSE A CHART: IF YOU WANT TO SHOW...

IF YOU WANT TO...	USE THIS CODE	WHAT IT DOES / REMEMBER
Compare categories with a bar chart	<code>result = df.groupby("Category")["Sales"].sum()</code> <code>result.plot(kind="bar")</code> <code>plt.title("Sales by Category")</code> <code>plt.xlabel("Category")</code> <code>plt.ylabel("Sales")</code> <code>plt.show()</code>	Best for comparing groups/categories.
See a numeric distribution	<code>plt.hist(df["Sales"], bins=10)</code> <code>plt.title("Distribution of Sales")</code> <code>plt.xlabel("Sales")</code> <code>plt.ylabel("Frequency")</code> <code>plt.show()</code>	Histogram shows how values are distributed.
See relationship between two numbers	<code>plt.scatter(df["Sales"], df["Profit"])</code> <code>plt.title("Sales vs Profit")</code> <code>plt.xlabel("Sales")</code> <code>plt.ylabel("Profit")</code> <code>plt.show()</code>	Scatter plots are useful for relationships/correlation.
Check spread and possible outliers	<code>plt.boxplot(df["Sales"].dropna())</code> <code>plt.title("Sales Boxplot")</code> <code>plt.ylabel("Sales")</code> <code>plt.show()</code>	Boxplots help show spread and outliers.

TENSORORBIT EDA BOOTCAMP - STUDENT CODE CHEAT SHEET

Page 4 - Useful extras

8. USEFUL EXTRAS

IF YOU WANT TO...	USE THIS CODE	WHAT IT DOES / REMEMBER
Show a trend over time	<pre>daily = df.groupby("Order_Date")["Sales"].sum() daily.plot(kind="line") plt.title("Sales Over Time") plt.xlabel("Date") plt.ylabel("Sales") plt.show()</pre>	Use for dates/time trends. Convert the date column first.
Make text lowercase	<pre>df["Category"].str.lower()</pre>	Useful when text formatting is inconsistent.
Find rows containing a word	<pre>df[df["Product"].str.contains("Laptop", case=False, na=False)]</pre>	Searches inside text values.
Extract month from a date	<pre>df["Month"] = df["Order_Date"].dt.month</pre>	Date must already be converted with <code>pd.to_datetime()</code> .
Rename a column	<pre>df.rename(columns={"Old_Name": "New_Name"}, inplace=True)</pre>	Useful when a column name is confusing.
Create a new column	<pre>df["Profit_Margin"] = (df["Profit"] / df["Sales"]) * 100</pre>	Creates a calculated variable.
Export your cleaned data	<pre>df.to_csv("cleaned_data.csv", index=False)</pre>	Saves your current DataFrame as a CSV file.

TENSORORBIT EDA BOOTCAMP - STUDENT CODE CHEAT SHEET

Page 5 - What to do when something happens

9. IF THIS HAPPENS...

IF YOU WANT TO...	USE THIS CODE	WHAT IT DOES / REMEMBER
You get KeyError: column not found	<code>print(df.columns)</code>	Check spelling, spaces and capital letters in the column name.
Your numeric calculation fails	<code>df.info()</code>	Check whether the column is actually numeric.
A column should be numeric but is text	<code>df["Sales"] = pd.to_numeric(df["Sales"], errors="coerce")</code>	Invalid values become missing (NaN), so check them afterward.
Your chart has missing-value problems	<code>df["Sales"].dropna()</code>	Use <code>.dropna()</code> on the column when appropriate.
You want to know what values exist	<code>df["Category"].unique()</code>	Shows unique values.
You want to know how many values exist	<code>df["Category"].nunique()</code>	Counts unique values.
You changed df and want to check it	<code>df.head()</code> <code>df.info()</code>	Re-check the dataset after cleaning or creating columns.

10. SIMPLE CAPSTONE WORKFLOW

IF YOU WANT TO...	USE THIS CODE	WHAT IT DOES / REMEMBER
Step 1 - Load	<code>df = pd.read_csv("your_file.csv")</code>	Get the dataset into Python.
Step 2 - Inspect	<code>df.head()</code> <code>df.shape</code> <code>df.info()</code> <code>df.describe()</code>	Understand what you are working with.
Step 3 - Clean	<code>df.isnull().sum()</code> <code>df.duplicated().sum()</code>	Find data-quality issues, then decide how to handle them.
Step 4 - Ask a business question	<code># Example: Which category has the highest sales?</code>	Your analysis should answer a real question.
Step 5 - Analyse	<code>df.groupby("Category")</code> <code>["Sales"].sum().sort_values(ascending=False)</code>	Use Pandas/NumPy to get evidence.
Step 6 - Visualise	<code>df.groupby("Category")["Sales"].sum().plot(kind="bar")</code> <code>plt.show()</code>	Use a chart that helps communicate the finding.
Step 7 - Explain	<code># Finding: ...</code> <code># Recommendation: ...</code>	Do not stop at code. Explain what the result means for the business.

REMEMBER: You do NOT need to memorise the code. Choose the task you want to do, find it on this sheet, copy the pattern, replace the column names, run it, and explain the result.